

Resolución: Actividad 3 - Escape del Laboratorio

Qué problema resuelve la actividad.

Desafío 1: Reduce un número de múltiples dígitos a un solo dígito de manera recursiva mediante la suma sucesiva de sus componentes.

Desafío 2: Encuentra el número de mayor valor dentro de un arreglo aplicando la técnica de divide y vencerás.

Desafío 3: Encuentra la ruta más corta entre un punto de inicio (S) y un punto de llegada o meta (T) dentro de una matriz bidimensional llena de obstáculos, utilizando el algoritmo de backtracking.

Cómo funciona cada desafío.

Desafío 1: Toma un número entero y extrae sus dígitos sumando el residuo de la división entre 10 ($n \% 10$) con el resultado de la llamada recursiva aplicada a la división entera del número entre 10 ($\text{count}(n / 10)$). Si la suma resultante es mayor o igual a 10, vuelve a llamar a la función de manera recursiva sobre la suma hasta reducir el valor a un solo dígito ($n < 10$), el cual activa el caso base.

Desafío 2: Divide recursivamente el arreglo de números por la mitad calculando un punto medio ($\text{middle} = (\text{start} + \text{end}) / 2$). Cada subarreglo continúa dividiéndose hasta llegar a subproblemas triviales de un solo elemento. En la fase de combinación, compara los máximos devueltos por la mitad izquierda y la mitad derecha utilizando $\text{Math.max}(\text{leftMax}, \text{rightMax})$, propagando el valor más grande hacia arriba hasta obtener el máximo global del arreglo.

Desafío 3: Realiza una exploración en profundidad sobre la matriz probando desplazamientos a las cuatro celdas vecinas (arriba, abajo, izquierda, derecha). En cada paso válido, marca la casilla con 'V' para evitar ciclos. Si llega a la meta 'T', guarda la ruta si es la primera encontrada o si es más corta que la anterior mejorCamino. Al topar con pared, casillas visitadas o un camino que ya superó la longitud de la mejor ruta encontrada, deshace el movimiento actual (restaurando el carácter original de la celda y sacándolo de la lista camino) para explorar otras ramas.

Qué técnica se utilizó.

Desafío 1: Recursión Simple (evaluación recursiva directa sobre la pila de llamadas).

Desafío 2: Divide y Vencerás.

Desafío 3: Backtracking.

Cómo ejecutar el programa

El programa se ejecuta en la terminal de comandos (cmd) utilizando la instrucción: `java laberinto < maze.txt` (en ocasiones se pueden presentar problemas de lectura local según la configuración del entorno).

Cómo se obtiene la solución.

Desafío 1 (Raíz Digital): Se obtiene al ir acumulando los dígitos mediante divisiones entre 10 ($n / 10$) y residuos ($n \% 10$) en llamadas recursivas. Cada suma se vuelve a evaluar hasta reducir el número a un solo dígito (menor a 10), el cual se retorna como resultado final.

Desafío 2 (Valor Máximo): Se obtiene dividiendo el arreglo recursivamente a la mitad hasta tener subarreglos de un solo elemento. Al regresar de las llamadas recursivas, la función `Math.max()` compara los valores de la mitad izquierda y derecha, devolviendo el número más grande hacia la llamada principal.

Desafío 3 (Laberinto): Se obtiene explorando las 4 direcciones válidas y construyendo una lista con las coordenadas de la ruta actual (camino). Cuando el algoritmo alcanza la casilla de la meta 'T', evalúa si la ruta encontrada es la primera o si es más corta que la guardada anteriormente (`mejorCamino`). De ser así, guarda una copia de esta ruta y, al finalizar la exploración de todo el laberinto, imprime la lista de coordenadas en el formato `UTA:r0,c0;r1,c1`.

Qué decisiones importantes tomaste durante la implementación.

Se decidió agregar una condición para interrumpir la búsqueda en cualquier rama donde la longitud del camino actual sea mayor o igual a la de la mejor ruta encontrada hasta ese momento (`camino.size() >= mejorCamino.size()`). Esto evita explorar llamadas recursivas innecesarias y resuelve problemas de ejecución infinitos o muy lentos.

En lugar de dejar las casillas marcadas como visitadas de forma permanente, se decidió restaurar el carácter original de la matriz (`laberinto[nr][nc] = original`) al regresar de una llamada recursiva. Esto permite que una misma celda pueda ser parte de otros caminos potencialmente más cortos en futuras exploraciones.

Pruebas

Entrada utilizada: 351, 947.

```

public class actividad3 {

    //desafio 1
    static int count(int n){
        if(n<10){
            return n;
        }
        int suma=(n%10)+count(n/10);
        return count(suma);
    }

    //desafio 2
    static int max(int[] array, int start, int end){
        if(start == end){
            return array[start];
        }
        int middle = (start + end) / 2;

        int leftMax = max(array, start, middle);
        int rightMax = max(array, middle + 1, end);

        return Math.max(leftMax, rightMax);
    }
}

```

```

3
5
1
947

```

Casos normales:

- **Casos normales.**

. Llegada exitosa a la meta y guardado de la primera ruta

```
// PASO 1. Si (r,c) es la meta:
if (r == tr && c == tc) {
    if (mejorCamino == null || camino.size() < mejorCamino.size()) {
        mejorCamino = new ArrayList<>(camino);
    }
    return;
}
```

Avance y exploración estándar de celdas

```
// PASO 3. Moverse (backtracking)
char original = laberinto[nr][nc];
laberinto[nr][nc] = 'v';
camino.add(new int[]{ nr, nc });
System.out.println(nr + "," + nc);

buscar(nr, nc, tr, tc, camino);
```

Desmarcado y retroceso tras explorar un camino

```
camino.remove(camino.size() - 1);
laberinto[nr][nc] = original;
```

Casos límite o especiales:

- Fuera de los límites del mapa

```
// Si queda FUERA del laberinto, continúe.
if (nr < 0 || nr >= filas || nc < 0 || nc >= columnas) {
    continue;
}
```

- Evitar ciclos e influir en paredes

```
// Si es pared '#' o ya visitado 'v', continúe.
if (laberinto[nr][nc] == '#' || laberinto[nr][nc] == 'v') {
    continue;
}
```

Evidencia de que la ruta encontrada es realmente la más corta



Explicación técnica

¿Cuál es el caso base del desafío 1?

El caso base del desafío 1 ocurre cuando el número evaluado n se reduce a un solo dígito, es decir, cuando es estrictamente menor a 10 ($n < 10$).

¿Cuál es el caso recursivo?

El caso recursivo es la parte de la función que se ejecuta cuando el problema aún no ha alcanzado el caso base.

¿Cómo se aplica divide y vencerás en el desafío 2?

La técnica de divide y vencerás se aplica descomponiendo el problema de búsqueda mediante una estructura lógica en forma de árbol binario de decisión.

¿Cómo se divide el problema?

Se calcula el índice central de la secuencia de datos para partir el problema exactamente a la mitad. Este proceso genera dos ramas o subproblemas independientes en el árbol de decisión: una para la mitad izquierda y otra para la mitad derecha. Cada subproblema se vuelve a dividir de forma idéntica en llamadas descendentes por el árbol.

¿Cómo se combinan los resultados?

Una vez resueltas las hojas, el algoritmo asciende por el árbol de decisión comparando los resultados parciales devueltos por la rama izquierda y la rama derecha. Se selecciona el candidato de mayor valor descartando el menor, propagando el ganador hacia los niveles superiores hasta consolidar el máximo global en la raíz del árbol.

¿Cómo funciona el backtracking del desafío 3?

El backtracking del desafío 3 explora de manera exhaustiva el laberinto mediante búsqueda en profundidad. Desde la celda actual, prueba secuencialmente avanzar a las cuatro direcciones vecinas (arriba, abajo, izquierda, derecha). Si una celda vecina es válida, se marca como visitada 'V' y se agrega a la lista camino para continuar la búsqueda recursiva. Si la ruta llega a un callejón sin salida o procesa una rama completa, el algoritmo deshace el último movimiento: remueve la celda de camino, le restaura su carácter original en el mapa y regresa al estado anterior para probar direcciones alternativas.

¿Cómo evitas ciclos?

Los ciclos en el desafío 3 se evitan marcando temporalmente las casillas que forman parte de la ruta actual.

¿Cómo determinas cuál ruta es la más corta?

Cada vez que el algoritmo llega a la celda objetivo ('T'), verifica la cantidad de pasos o casillas acumuladas en la lista del camino actual (`camino.size()`). Si ya existía una ruta previa en `mejorCamino`, se compara el número de celdas: si la nueva lista tiene una cantidad menor de elementos (`camino.size() < mejorCamino.size()`), se reemplaza `mejorCamino` con la nueva solución. Al acabar la exploración, la lista retenida en `mejorCamino` corresponde al camino con el menor número de movimientos.

¿Qué optimización aplicaste y por qué?

Durante el recorrido recursivo, antes de continuar explorando las direcciones de una celda, el algoritmo evalúa si la longitud de la ruta actual (`camino.size()`) ya es igual o mayor al número de pasos de la mejor ruta encontrada hasta ese momento (`mejorCamino.size()`). Si esta condición se cumple, la función cancela inmediatamente esa rama con un `return`. Se aplicó porque evita exploraciones inútiles si ya existe un camino válido de, por ejemplo, 10 pasos a la meta, reduciendo drásticamente el espacio de búsqueda.

Reflexión final

¿Qué diferencia existe entre una solución recursiva y una solución iterativa?

La diferencia principal radica en la estructura de control y la gestión de memoria del sistema: la recursividad resuelve un problema haciendo que la función se llame a sí misma con subproblemas más pequeños, delegando el control del estado y las variables a la pila de llamadas del sistema; por su parte, la solución iterativa resuelve el problema utilizando ciclos explícitos de control (`for`, `while`) y estructuras de datos administradas directamente por el programador (como arreglos, pilas o colas).

¿Por qué divide y vencerás puede facilitar la resolución de ciertos

problemas?

Porque permite descomponer un problema grande y difícil en subproblemas pequeños, idénticos y más sencillos de resolver. Además, al dividir los datos sucesivamente (generalmente a la mitad), disminuye drásticamente el número de operaciones necesarias en comparación con los enfoques iterativos directos.

¿Por qué encontrar una solución con backtracking no significa necesariamente que sea la mejor solución?

No garantiza que sea la mejor porque el algoritmo se limita a buscar cualquier ruta válida que cumpla con las condiciones para llegar a la meta, deteniéndose o registrando el primer camino que logra completarse con éxito.

¿Qué problema podría presentarse si el algoritmo no controla las posiciones ya visitadas?

Al no marcar las celdas por las que ya pasó, el explorador podría moverse de una casilla a otra en bucle. Esto generaría una cadena interminable de llamadas recursivas que consumiría toda la memoria disponible del sistema hasta colapsar el programa.

¿Qué aprendiste al tener que encontrar la ruta más corta en lugar de simplemente encontrar una ruta?

Aprendí que buscar un resultado óptimo en lugar de simplemente cualquier solución válida exige un nivel superior de control lógico y estructural. Mientras que hallar una ruta cualquiera solo requiere verificar que exista un camino disponible, garantizar la optimización requiere un manejo estricto del estado y de las estructuras de control. además me di cuenta que mi computadora no aguanta esto